

Software Testing and Validation (CCSW 323)

Semester II- 2026

Analysis and Testing of WordPress (wp-comments-post.php)

Supervised by: Dr Fatimah Assiri

Section [A]

Group 1

Submitted by:

Name	Student ID
Reman Alamoudi	2312582
Ghaida Almehmadi	2317174
Mariam Anbar	2310434
Lama Suliman	2311250
Raghad Ali Alghamdi	2312473

Table of Contents

Part 1: Software Selection and Testing	3
Software Under Test.....	3
Brief Description.....	3
GitHub Repository Link:	3
Size Of The Software.....	3
Software Under Test:	3
Selected Model and Coverage Criteria	4
Testing Model	4
Coverage Criteria	4
Graph:	5
Generated Test cases	6
Test Cases.....	7
Functionality/Part of the Software	7
Test Cases (split per coverage criteria)	7
Part 2: Tool Application	9
Introduction to the Tool (Postman)	9
Tool Description.....	9
Postman Key features:	9
The purpose of using Postman	9
Tool setup and Configuration.....	10
Tool Usage	10
Application of the Tool	10
Example Test Case Execution.....	10
Testing Results	13
Output of Test cases	13
Conclusion	13

Part 1: Software Selection and Testing

Software Under Test

Brief Description

wp-comments-post.php file is a core WordPress script that handles and processes comment submissions. It will make sure to send requests via POST method, process and validate the data sent in comments, user cookie permission, triggers the relevant WordPress actions and safely redirect the user to the correct page on successful comment submission, which will require moderation of the information in some cases.

GitHub Repository Link:

<https://github.com/WordPress/WordPress/blob/master/wp-comments-post.php>

Size Of The Software

- Lines of Code (LOC) (excluding comments and documentation): 45
- Number of classes: 0
- Number of methods per class: 0

Note: This file yields 0 classes and 0 methods because it is a procedural script. It only calls existing WordPress functions and does not declare any new classes or methods.

Software Under Test:

The system we have chosen to test is the wp-comments-post.php file because of the complexity of the full WordPress core. And this module holds the key logic to do user interactions and is adequate to show our testing models and coverage criteria.

Selected functionalities:

1. Request Validation
2. Comment Submission
3. Moderation Handling
4. Cookie Consent

These four particular functionalities were selected since they are the main logical paths and decision points in the script. To be sure that only POST requests are accepted to preserve security, Request Validation was chosen, and Comment Submission will enable us to verify that the data is being processed as per the user input. Moreover, Moderation Handling is important in the decision of whether to publish or keep a comment under consideration, and Cookie Consent provides the system with user privacy. When these areas are tested concurrently, we can be able to check the integrity of data, flow and proper functioning of the system in all the important execution paths.

Selected Model and Coverage Criteria

Testing Model

1. Input Space Partitioning

We will be using Input Space Partitioning (ISP) to test WordPress' wp-comments-post.php file core functionality. ISP uses the input space to test the software; it divides possible inputs into different partitions that have similar predictable output that can then be used to test functionality.

The input space domain was partitioned into 3 sets:

- Request Type (POST, GET/Non-POST)
- Comment Content (Valid, invalid/empty)
- Cookie Consent (Given, Not Given)

Moderation handling is excluded from ISP because it is triggered by valid input combinations rather than direct input. Instead, it is generated after executing the comment validation path if the comment is valid.

Input Domain Model (IDM) table:

Characteristic	B1	B2
Request Type	POST	GET/Non-POST
Comment Content	Valid	Invalid/empty
Cookie Consent	Given	Not Given

2. Graph-Based Testing Model

The second testing model we choose is Graph-based testing model. This model is represented as a set of nodes (statements) and edges (flow path between statements) of the software. It describes execution path followed by the software which is important to ensure that the software is fully functioning.

Coverage Criteria

1. Each Choice Coverage (ECC)

Each Choice Coverage was chosen based on the ISP model. In ECC we ensured that each input choice is used at least once to ensure the maximum possible coverage criteria is reached, which increases the validity of the tests.

2. Edge Coverage

We choose Edge covers since it guarantees that each transition between nodes in the software will be carried out at least once. As a result, we can cover more test requirements using less test cases. Edge Coverage is applied to this system to the logical flow of the comment submission process, which includes:

- Request validation
- Comment Submission
- Cookie Consent
- Moderation handling (generated after the execution of comment submission if the comments happened to be valid.)

Graph:

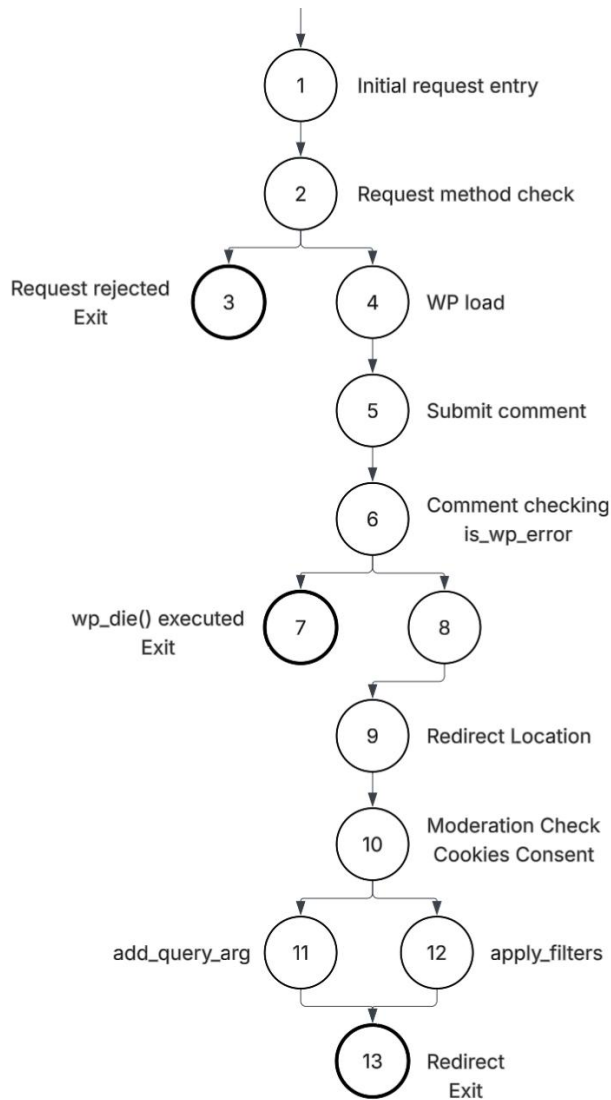


Table:

Test Path	[1, 2, 4, 5, 6, 8, 9, 10, 12 ,13] [1, 2, 4, 5, 6, 8, 9, 10, 11 ,13] [1, 2, 4, 5, 6, 7] [1, 2, 3]
TR	[1, 2] [2, 3] [2, 4] [4, 5] [5, 6] [6, 7] [6, 8] [8, 9] [9, 10] [10, 11] [10, 12] [11, 13] [12, 13]

Generated Test cases

1. Each Choice Coverage Test Cases

Test Cases	Test values	Expected Output
TC1	Request Type: POST Comment: Valid Cookie: Given	Comment accepted Moderation approved Cookie stored
TC2	Request Type: POST Comment: Valid Cookie: Not Given	Comment accepted Moderation is triggered, comment is placed under review No cookie stored
TC3	Request Type: POST Comment: Invalid/Empty Cookie: Given	Comment rejected (validation failure) No moderation processing Cookie not stored
TC4	Request Type: GET/Non-POST Comment: Invalid/Empty (or ignored) Cookie: Not Given	Request rejected (405 Method Not Allowed) No comment processing No cookie stored No moderation processing

Coverage Check:

Test Cases	Request Type	Comment Content	Cookie Consent
TC1	POST	Valid	Given
TC2	POST	Valid	Not Given
TC3	POST	Invalid	Given
TC4	GET/Non-POST	Invalid	Not Given

2. Edge Coverage Test Cases

Test Cases	Test Scenario	Test values	Expected
TC5	Submit a valid comment with authentication cookie	Request Type: POST Comment: Valid Cookie: Given	Comment accepted Comment passes validation Moderation logic applied Cookie Stored
TC6	Submit a valid comment without authentication cookie	Request Type: POST Comment: Valid Cookie: Not Given	Comment accepted System applies moderation logic Request is processed according to moderation rules
TC7	Submit an empty comment	Request Type: POST Comment: Invalid/Empty Cookie: Given	Request rejected Validation error is returned
TC8	Send request using invalid HTTP method	Request Type: GET/Non-POST Comment: Invalid/Empty (or ignored) Cookie: Not Given	Request rejected Invalid request response returned

Test Cases

Functionality/Part of the Software

The analysed software component is Wp-comments-post.php, the main WordPress file in charge of managing and verifying user generated comments. There are multiple logical stages to its functionality:

- Request Validation: By examining the HTTP request method, the system serves as a gatekeeper. It ensures that data is submitted safely and in accordance with protocol by only allowing POST request.
- Core Environment Loading: By loading wp-load.php, it connects to the WordPress core and enables the script to use security headers, database operations, and user authentication.
- Data Processing & Error Handling: The program verifies raw input data by comparing it to predefined guidelines. It creates a wp_error and ends execution if validation is unsuccessful.
- Moderation & Redirect Logic: The system evaluates the comment's status. If a comment is held for moderation, it ensures the user can see their "Awaiting Moderation" notice by dynamically changing the redirection URL to include a security hash.
- Failure Handling: If the comment is determine to be valid the system interrupts the normal flow. It uses the wp_die() function to initiate a failure response rather than rerouting.

Test Cases (split per coverage criteria)

1. Each Choice Coverage

Test Case ID	Test Inputs	Expected Output	Actual Output (After execution of tests)
TC1	Method: POST Comment: Valid ("Hello") Cookie: Given	Accepted comment, Cookie stored, standard redirect	The comment is successfully submitted And displayed in the comments section
TC2	Method: POST Comment: Valid comment ("Hello TC2") Cookie: Not given	Comment accepted, no cookie stored, and moderation hash added to redirect	The comment didn't appear publicly and went to moderation queue at WordPress admin comments
TC3	Method: POST Comment: Empty Cookie: Given	Comment rejected due to validation failure; execution terminated via error handling	Empty comment rejected due the validation failure and no comment created
TC4	Method: GET Comment: Ignored Cookie: Not Given	Request rejected (405 error)	GET request is rejected by the system with 405 not allowed response

2. Edge Coveragej

Test Case ID	Test Path	Test Inputs	Expected Output	Actual Output (After execution of tests)
TC5	[1, 2, 4, 5, 6, 8, 9, 10, 12 ,13]	Method: POST Comment: Valid ("Hello") Cookie: Given	Success path: Request is successfully processed and reaches successful completion at node 12 after passing validation and moderation decision stage	The POST request with valid comment and cookie was processed successfully. The comment passed validation, moderation, and execution reached completion path. The comment was displayed on the post page immediately
TC6	[1, 2, 4, 5, 6, 8, 9, 10, 11 ,13]	Method: POST Comment: Valid ("Hello TC6") Cookies: Not given	Moderation path: Reaches node 10 for validation and moderation decision stage, since cookies are not given and moderation is needed, it branches to node 11.	Valid comment with POST request and no cookie passed validation but triggered the moderation process queue and didn't display publicly
TC7	[1, 2, 4, 5, 6, 7]	Method: POST Comment: "" Cookie: Given	Validation error path: Detect an empty comment at node 6 and terminate at node 7 due to empty comment input.	Empty comment with POST request failed validation. The system terminated execution through error handling and no comment created
TC8	[1, 2, 3]	Method: GET Comment: Ignored Cookie: Not Given	Rejection path: Fails the method check at node 1 due to invalid HTTP method and terminate at node 2	The GET request is rejected during request method validation and it returned 405 method by not allowing response without processing any comment data

Part 2: Tool Application

Introduction to the Tool (Postman)

Tool Description

Postman is an API testing tool used to create and send HTTP requests. It is for validation backend functions by testing request handling, input validating and system response. Supporting multiple HTTP requests (GET, POST, PUT), response inspection (status code, headers, body) and execution logging.

Postman Key features:

Advanced API Client:

Supports all HTTP methods and provides different protocols compatibility including GraphQL, WebSockets, and SOAP.

Collections and Requests:

Enables organizing API requests into “collections,” which can include documentation, authorization details and saved responses for groups cooperation.

Automated Testing:

Delivering JavaScript-based scripting abilities to write tests that validate API responses, inspect status codes and execute integration testing.

Environments and Variables:

It enables the management of several sets of variables for the same request which facilitates and allows shifting between environments.

Monitors:

Continuously inspect API stability and performance by running collections on a scheduled basis in the cloud, with failures detection alerts.

Collaboration Features:

Provide shared workspaces that allow teams to cooperate in real time, including commenting and pull requests for version control.

Mock Servers:

Enable developers to emulate API endpoints before the actual backend is ready. This enables front-end development and testing to keep running without delaying and waiting for backend implementation.

The purpose of using Postman

Postman is used in this project to test the behavior of the wp-comments-post.php component by sending HTTP requests (such as POST and GET) direct to the server. The main goal is to validate how the system handles diverse inputs such as request type validation, comment submission and cookie consent cases. It

enables testers to examine system responses and observe how the application processes valid and invalid inputs and refusing unsupported methods.

Moreover, Postman supports the execution for the designed test cases based on Input Space Partitioning (ISP) and Graph-Based Testing models making sure of the accurate verification of coverage criteria without the need for a front-end interface.

Tool setup and Configuration

The tool setup requires configuring with a local WordPress environment created using LocalWP. Postman was sending requests to the WordPress endpoint wp-comments-post.php. Then the requests were configured by the POST and GET methods with request body formatted x-www-form-urlencoded to match WordPress comment submission requirement.

WordPress Admin discussion setting was needed to enable or disable manual configuration and moderation for some test cases.

Tool Usage

Application of the Tool

Postman was used to execute the designed test cases by sending HTTP request directly to the WordPress comments handling component with different HTTP request (GET, PUT, POST), comment content and cookie consent parameters were applied to test normal behaviour, error handling, moderation logic and request rejection scenarios. Each test case was implemented individually, and the system behaviour was observed through Postman response, WordPress website, and the admin panel dashboard.

Example Test Case Execution

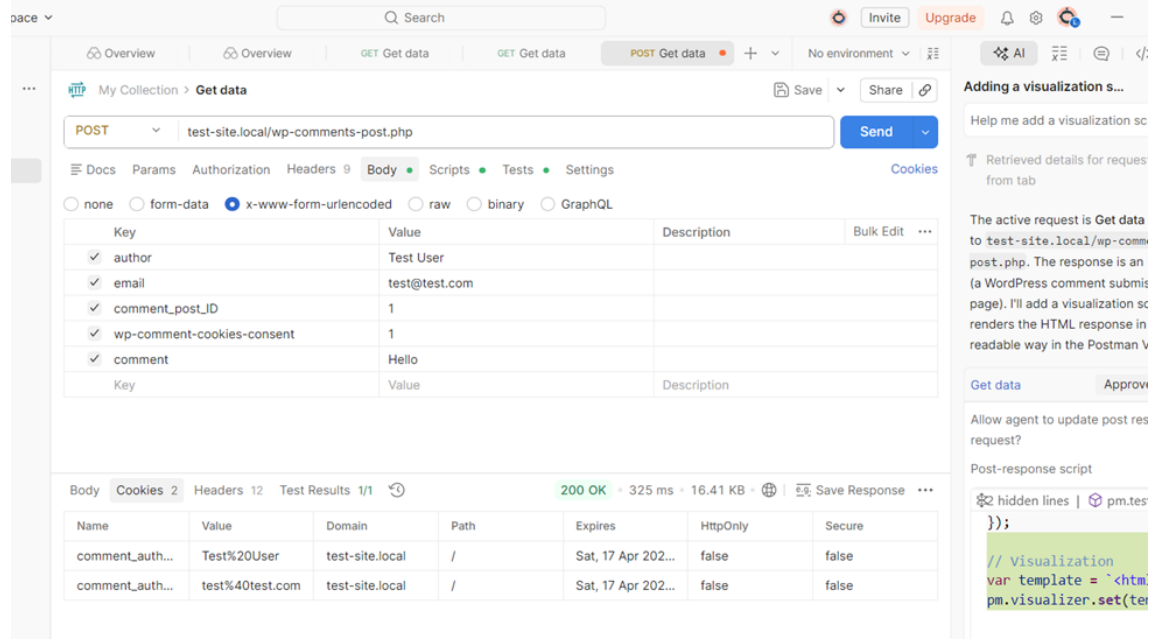
Test Case ID: TC1

A POST request was sent to wp-comments-post.php with a valid comment using Postman.

- The request was successfully processed.
- Cookies were stored.
- Comment displayed on the post page.

This execution confirmed the correct behaviour of the system for valid comment submission.

TC1 POST request to send Valid Comment Submission with cookies



The screenshot shows the Postman interface for a POST request to `test-site.local/wp-comments-post.php`. The request body is set to `x-www-form-urlencoded` and contains the following data:

Key	Value	Description
author	Test User	
email	test@test.com	
comment_post_ID	1	
wp-comment-cookies-consent	1	
comment	Hello	

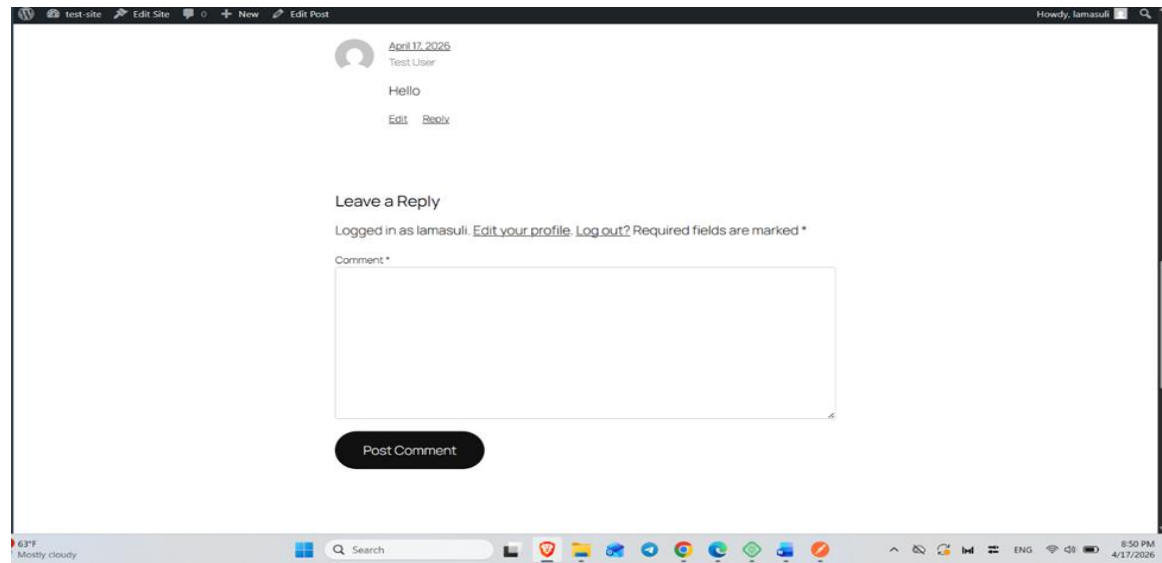
The response is a `200 OK` status with a response time of 325 ms and a size of 16.41 KB. The response body contains the following cookies:

Name	Value	Domain	Path	Expires	HttpOnly	Secure
comment_auth...	Test%20User	test-site.local	/	Sat, 17 Apr 202...	false	false
comment_auth...	test%40test.com	test-site.local	/	Sat, 17 Apr 202...	false	false

The right sidebar shows the "Adding a visualization s..." section with a "Get data" button and a "Post-response script" section containing the following code:

```
hidden lines | pm.test
});
// Visualization
var template = `<html>
pm.visualizer.set(tem
```

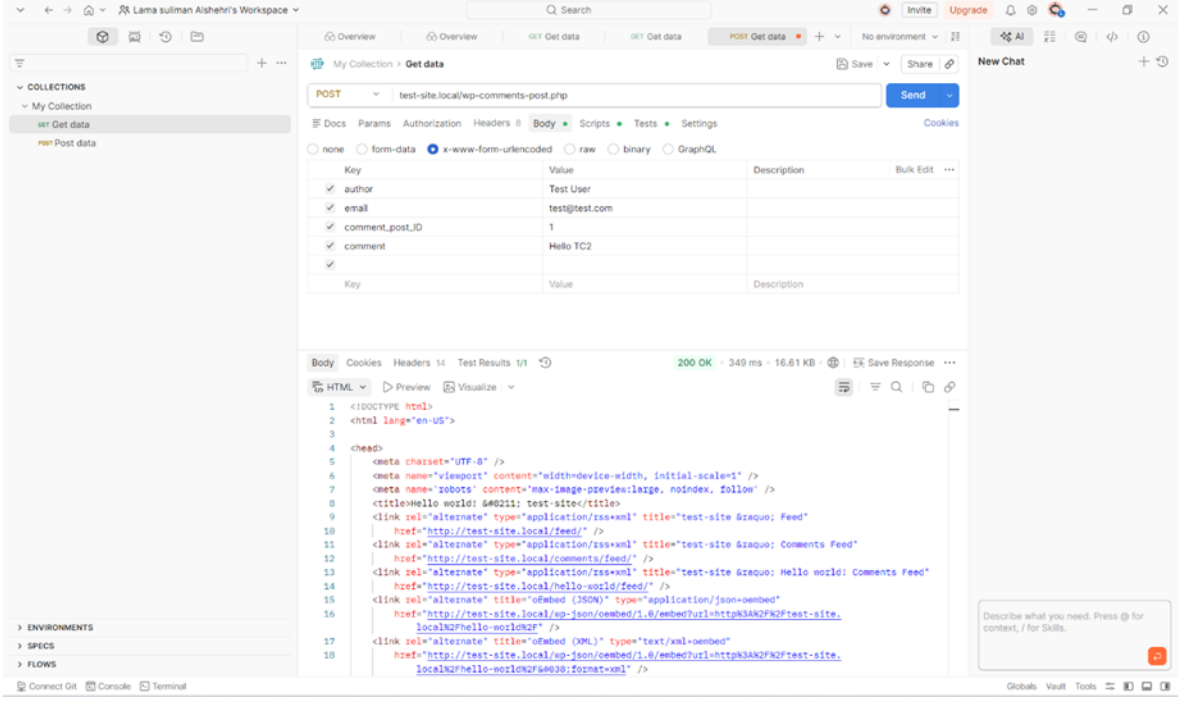
The request successfully processed, cookies were stored and the comment displayed on post page



The screenshot shows a web browser displaying a comment submission form. The form is titled "Leave a Reply" and includes a "Logged in as lamasuli. [Edit your profile](#) [Log out?](#) Required fields are marked *" message. The form contains a "Comment *" text area and a "Post Comment" button. Below the form, the comment is displayed as follows:

April 17, 2026
Test User
Hello
[Edit](#) [Reply](#)

TC2 Post request with a valid Comment Submission and no cookie given.

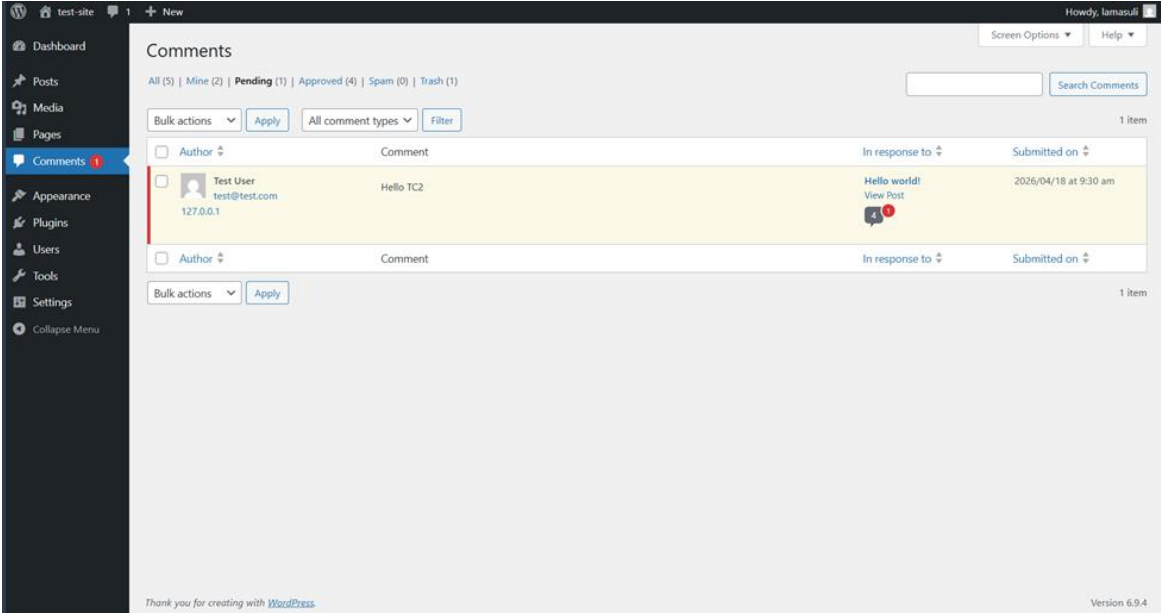


The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** test-site.local/wp-comments-post.php
- Body:** Form data (x-www-form-urlencoded) with the following fields:

Key	Value	Description
author	Test User	
email	test@test.com	
comment_post_ID	1	
comment	Hello TC2	
- Response:** 200 OK, 340 ms, 16.61 KB. The response body is an HTML document with a meta charset of UTF-8 and various meta tags for viewport, robots, and title. It includes links to feeds and a JSON feed.

The comment is accepted but the user information didn't store since the cookie was off.
WordPress add the comment in the moderation queue (pending) and did not display publicly.
The comment is valid but waits for admin approval.



The screenshot shows the WordPress dashboard with the 'Comments' section selected. The interface displays the following information:

- Comments List:** A table showing a single comment in a 'Pending' state.
- Comment Details:**

Author	Comment	In response to	Submitted on
☐ Test User test@test.com 127.0.0.1	Hello TC2		2026/04/18 at 9:30 am
- Actions:** Bulk actions and Apply buttons are visible at the top and bottom of the list.

Testing Results

Output of Test cases

The execution of the test cases provided the following outputs:

- TC1: Successful comment submission and immediately display for valid comment input
- TC2: Comment placement under moderation based on system moderation rules
- TC3: Validation error messages for empty comments
- TC4: 405 HTTP response to unsupported request methods

Total Number of Tests

Total of 4 Test Cases executed using the Postman tool. The same test set was used to evaluate the 8 test cases created by the two coverage criteria used: ECC and Edge Coverage. Due to overlap in input scenarios, the same test inputs were reused for both coverages.

Pass/Fail Results

Passed: 4 test cases -100%.

Failed: 0 test cases - 0%.

Conclusion

The testing process of the wp-comments-post.php component revealed that using a Postman tool was highly efficient in validating of the system's behavior under several input variables conditions. The tool enabled precise testing of main functionalities such as request validation, comment submission and moderation logic by sending HTTP requests straightforward to the server and analyzing the responses. Therefore, all designed test cases were executed successfully affirming that the system behaves correctly for each of valid and invalid scenarios, containing proper error handling and refusal of unsupported request methods.

Besides its efficiency, some challenges were faced during the testing process. During the testing process, multiple challenges were faced and one of those challenges was configuring the WordPress admin settings matching with the designed test cases specifically for features like comment moderation and cookie handling because required careful adjustments to make sure for accurateness the system behavior.

For an improved testing process, it is recommended to use automated test scripts like Postman's automated script to enhance efficiency and reduce manual effort. Also using Postman collections and environment more regularly can help to standardize request configurations and reduce setup errors. In addition, improved documentation of test cases and input data making sure better alignment between test design and execution and combining Postman with additional monitoring or logging tools could deliver deeper understanding into system behavior especially for backend processes like moderation handling.